

 [Cliquer ici pour Enregistrer en PDF](#)

Rapport de Déploiement : Architecture Backend TontineChain

Projet : TontineChain (Hackathon MIABE 2026)

Date : 3 Mai 2026

Statut : Déployé en Production

1. Présentation de l'Architecture

Le backend de **TontineChain** est une API RESTful robuste et sécurisée construite avec **Laravel 13** (PHP 8.2). Il a été conçu spécifiquement pour gérer les flux complexes d'une tontine numérisée intégrant des services financiers (Mobile Money) et une vérification par technologie Blockchain.

1.1 Infrastructure & Hébergement

L'infrastructure a été pensée pour être "Production-Ready" et hautement disponible :

- **Hébergeur :** Render (Cloud Platform).
- **Conteneurisation :** Docker avec environnement Apache optimisé (gestion stricte du DocumentRoot et des permissions du conteneur).
- **Base de Données :** PostgreSQL 16 managé par Render, garantissant la persistance et l'intégrité des transactions.
- **Déploiement Continu (CI/CD) :** Synchronisation automatique avec GitHub via un pipeline défini par le fichier `render.yaml`. Une routine d'automatisation gère l'exécution des migrations à chaque déploiement.

1.2 Intégrations Tiers (Services Externes)

Le backend sert de chef d'orchestre entre plusieurs technologies :

- **Passerelle de Paiement FedaPay :** Intégration pour la collecte des cotisations et la redistribution des fonds (Payouts) via Mobile Money. Gestion asynchrone sécurisée via Webhooks.

- **Communication Infobip** : Notification multi-canal (SMS & WhatsApp) pour l'authentification (OTP), les rappels d'échéances et la confirmation des transactions.
- **Contrat Intelligent Polygon (Blockchain)** : Preuve d'immuabilité pour les transactions, les paiements et le score de confiance.

2. Sécurité & Authentification

L'approche sécuritaire a été placée au centre du développement :

- **Passwordless Auth** : Remplacement des mots de passe traditionnels par une authentification par **OTP** (One Time Password) envoyée par SMS/WhatsApp.
- **Tokens Sanctum** : Gestion des sessions via Laravel Sanctum. Toutes les routes sensibles sont protégées par le middleware `auth:sanctum`.
- **Gestion des Erreurs** : Configuration stricte garantissant des retours JSON propres (401, 403) empêchant l'exposition des erreurs système.
- **Protection des Webhooks** : Validation cryptographique des signatures FedaPay pour prévenir les requêtes frauduleuses.

3. Modélisation Métier & Fonctionnalités Clés

L'API englobe toutes les règles de gestion d'une tontine moderne :

- **Gestion des Utilisateurs & Profils** : Calcul algorithmique d'un "Score de Confiance" basé sur l'historique financier et les éventuels incidents.
- **Gestion des Groupes de Tontine** : Création, invitation, adhésion, statistiques financières temps-réel et suivi des cycles.
- **Cotisations et Payouts** : Versement des fonds et automatisation algorithmique des ramassages.
- **Gouvernance (Enchères & Votes)** : Implémentation d'un système de permutation de positions (Votes) et d'un système d'enchères inversées.
- **Pénalités (Incidents)** : Système automatique de gestion des retards et amendes.

4. Documentation & Liens Utiles

Toutes les routes de l'API (soit **30 endpoints distincts**) ont été rigoureusement décrites à l'aide d'attributs PHP 8 pour la génération automatique de la documentation Swagger.

 **URL de Base de l'API :**

<https://tonnine-benin-backend.onrender.com/api/v1>

 **Documentation Swagger Interactive :**

<https://tonnine-benin-backend.onrender.com/api/documentation>

 **Test de Santé (Health Check) :**

<https://tonnine-benin-backend.onrender.com/api/v1/health>